AI BASED PROJECT PLANNING ASSISTANT

MINI PROJECT REPORT

Submitted by

AJAY M	**Register No.: 23TD0003**
SHAFI AHMED F	**Register No.: 23TD0076**
VIDYASAAGAR M	**Register No.: 23TD0091**

in partial fulfillment of the requirements for the degree
of
BACHELOR OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE and Affiliated to Pondicherry University)

Accredited with 'A+' Grade in Cycle II by NAAC

PUDUCHERRY-607402

MAY 2026

RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

PONDICHERRY UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that the mini project work entitled "**AI BASED PROJECT PLANNING ASSISTANT**" is a bonafide work done **AJAY M [REGISTER No.:23TD0003], SHAFI AHMED F [REGISTER No.: 23TD0076], VIDYASAAGAR M [REGISTER No.: 23TD0091]** in partial fulfillment of the requirement, for the award of B.Tech Degree in Computer Science and Engineering by Pondicherry University during sixth semester in the academic year 2025-2026.

PROJECT COORDINATOR **HEAD OF THE DEPARTMENT**

Date of Examination: ….....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We are very thankful and grateful to our beloved guide, **Dr. R.G. SURESH KUMAR** whose great support in valuable advices, suggestions and tremendous help enabled us in completing our project. He has been a great source of inspiration to us.

We also sincerely thank our Head of the Department, **Dr. R.G. SURESH KUMAR** whose continuous encouragement and sufficient comments enabled us to complete our project report.

We thank all our **Staff members** who have been by our side always and helped us with our project. We also sincerely thank all the lab technicians for their help as in the course of our project development.

We would also like to extend our sincere gratitude and grateful thanks to our principal **Dr. E. VIJAYAKRISHNA RAPAKA** for having extended the Research and Development facilities of the department.

We are grateful to our Chairman **Shri. M. K. RAJAGOPALAN**. He has been a constant source of inspiration right from the beginning.

We wish to thank our **family members and friends** for their constant encouragement, constructive criticisms and suggestions that has helped us in timely completion of this project.

DECLARATION

We hereby certify that the work which is being presented in the project entitled "AI BASED PROJECT PLANNING ASSISTAN" submitted in the DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING, RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY is an authentic record of my own work carried under the supervision of Dr. R. G. SURESH KUMAR Professor and Head of the Department. This study has not been submitted to any other institution or university for the award of any other degree.

Signature of the Student

ABSTRACT

BuildFlow AI is an academic and practical software engineering project developed to address a recurring problem faced by student teams, startup founders, and independent developers during the early stages of software project planning. In many educational and real-world settings, project planning is performed in an unstructured manner. Ideas are often discussed informally, requirements are captured inconsistently, and critical planning documents such as Business Requirements Documents (BRD), Software Requirements Specifications (SRS), development roadmaps, and implementation task lists are created manually.

This manual approach consumes significant time, introduces ambiguity, and frequently results in incomplete or poorly aligned planning outputs. Generic conversational artificial intelligence tools can assist with brainstorming, but they do not typically enforce a structured multi-stage planning process, maintain project-specific memory in a controlled workflow, or automatically transform gathered inputs into reusable planning artifacts.

BuildFlow AI proposes a focused solution in the form of an AI-based project planning assistant that guides users through a stage-wise planning workflow. The system uses a React-based frontend for user interaction, a Node.js and Express backend for application logic and AI request orchestration, Firebase for authentication and cloud data persistence, and an AI API integration through OpenRouter to generate structured responses and planning documents.

The platform collects project information across successive stages such as idea definition, target users, core features, user interface preferences, and technology stack. It then stores the generated blueprint and uses that blueprint to create meaningful outputs including BRD, SRS, roadmap documents, and implementation-oriented task prompts.

The project demonstrates how domain-focused AI assistance can be made more reliable by constraining the interaction flow, enforcing structured response formats, and coupling conversational intelligence with persistent project memory. The outcome of the system is a planning environment that reduces documentation effort, improves clarity of project requirements, increases consistency between early inputs and final outputs, and provides a reusable academic and practical foundation for software teams.

BuildFlow AI therefore functions not merely as a chatbot, but as a guided planning assistant that converts scattered ideas into organized and actionable project documentation.

.

LIST OF TABLES

TABLE NO.	NAME OF THE TABLE	PAGE NO.
3.1	Software Requirements	7
3.2	Hardware Requirements	8
3.3	Feasibility Analysis	8
5.1	Module Responsibility Mapping	20
5.2	Algorithms and Techniques Used	21
6.1	Functional Testing Coverage	25
6.2	Detailed Test Cases	26

LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO.
4.1	System Architecture	10
4.2	Data Flow Diagram (Level 0)	11
4.3	Data Flow Diagram (Level 1)	14
4.4	Use Case Diagram	16
4.5	Class Diagram	17
4.6	Activity Diagram	18
4.7	Sequence Diagram	19
7.1	Landing Page Interface	28
7.2	User Registration Interface	28
7.3	User Login Interface	29
7.4	Dashboard Interface	29
7.5	Create Project Interface	30
7.6	Workspace – Chatbot Interaction	30
7.7	Stage-Based Planning Interface	31
7.8	Blueprint Panel Interface	31
7.9	Output Panel Interface	32
7.10	Prompt Generation Interface	32
7.11	Settings Interface	33
7.12	Light and Dark Theme Interface	

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
UI	-	User Interface
UX	-	User Experience
LLM	-	Large Language Model
API	-	Application Programming Interface
MVP	-	Minimum Viable Product
JSON	-	JavaScript Object Notation
SPA	-	Single Page Application
HTTP	-	HyperText Transfer Protocol
MVC	-	Model-View-Controller
BRD	-	Business Requirements Document
SRS	-	Software Requirements Specification
DFD	-	Data Flow Diagram
URL	-	Uniform Resource Locator

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	iv
	LIST OF FIGURES	v
	LIST OF ABBREVIATIONS	vi
1.	**INTRODUCTION**	**1**
	1.1 Motivation	
	1.2 Problem Definition	
	1.3 Project Objective	
	1.4 Project Scope	
2.	**LITERATURE SURVEY**	**4**
	2.1 Existing System	
	2.2 Limitations of Existing System	
	2.3 Proposed System	
3.	**SYSTEM ANALYSIS & REQUIREMENTS**	**7**
	3.1 Software Requirements (e.g., OS, Python/Java, Databases)	
	3.2 Hardware Requirements	
	3.3 Feasibility Study	
4.	**SYSTEM DESIGN**	**10**
	4.1 System Architecture	
	4.2 Data Flow Diagram (DFD)	
	4.3 UML Diagrams (Use Case, Class, Activity, Sequence,etc)	

5.	**IMPLEMENTATION**	**20**
	5.1 Modules Description	
	5.2 Algorithms/Techniques Used	
	5.3 Key Code Snippets	
6.	**TESTING**	**25**
	6.1 Testing Methods (Unit, Integration,etc)	
	6.2 Test Cases	
7.	**RESULTS & DISCUSSION**	**27**
	7.1 Screenshots/Output Interface	
8.	**CONCLUSION & FUTURE ENHANCEMENT**	**35**
9.	**REFERENCE**	**37**
10.	**APPENDIX**	**38**

CHAPTER 1
INTRODUCTION

The increasing complexity of software projects makes structured planning essential before development begins. Teams must clearly define objectives, users, features, and technical approaches to ensure successful execution. However, in academic environments, students often lack a systematic method to convert ideas into proper documentation and actionable plans.

BuildFlow AI is designed to address this gap by acting as an AI-based project planning assistant. Instead of functioning as a general chatbot, it guides users through structured stages such as idea definition, user analysis, feature planning, UI/UX design, and technology selection. The system collects and maintains context throughout these stages and generates structured outputs like BRD, SRS, and development roadmaps. By combining a React-based frontend, Node.js backend, AI integration, and Firebase storage, the system ensures continuity, usability, and accurate project-specific outputs. This approach helps users transform abstract ideas into well-defined and implementable project plans.

1.1 Motivation

The motivation for BuildFlow AI arises from the difficulty students and small teams face during the early stages of project planning. While ideas are often available, converting them into structured documents, clear requirements, and development plans is challenging. Traditional approaches rely on manual notes and informal discussions, leading to inconsistency, delays, and incomplete planning.

Although AI tools can assist in content generation, they often produce generic or disconnected outputs when used without structure. This highlights the need for a guided system that collects project-specific information in a systematic way.

BuildFlow AI addresses this by combining structured workflows with AI capabilities. It helps users move step-by-step from idea to implementation planning, ensuring clarity, consistency, and completeness. Additionally, it supports better learning by encouraging a systematic approach to software development, aligning with academic and engineering practices.

1.2 Problem Definition

The problem addressed by BuildFlow AI can be formally defined as follows: software project teams lack a structured, efficient, and project-specific mechanism for transforming early ideas into complete planning documentation and actionable implementation guidance.

The problem has multiple dimensions:

1. Manual planning is slow and often fragmented.
2. Existing generic AI tools are not specialized for staged project planning.
3. Teams struggle to maintain continuity between discussions and formal documents.
4. Generated requirements and plans are often inconsistent with the original project goal.
5. Academic teams need formal outputs such as BRD and SRS, but creating them manually is time-consuming and error-prone.

In the case of BuildFlow AI, the problem is not simply "users need AI help." Rather, users need guided planning support that combines conversational interaction, memory of earlier stages, structured data capture, and document generation. The system therefore defines the problem as one of workflow design and knowledge transformation. It must convert unstructured user input into stage summaries, preserve those summaries as a blueprint, and generate downstream outputs from that blueprint.

1.3 Project Objective

The primary objective of BuildFlow AI is to design and develop an AI-based project planning assistant that helps users generate structured planning documents and development guidance through a stage-wise interactive workflow.

The specific objectives are:

1. To create a user-friendly web application for project planning support.
2. To implement authentication and cloud-based project storage for personalized planning sessions.
3. To divide project planning into clearly defined stages that reduce ambiguity and improve completeness.
4. To integrate an AI service capable of asking relevant follow-up questions and generating structured responses.
5. To preserve stage summaries as a reusable project blueprint.
6. To generate planning outputs such as BRD, SRS, roadmap, and TODO documents using stored blueprint data.
7. To generate categorized task prompts for development, UI, backend, and deployment work.
8. To reduce planning time while improving document quality and consistency.

These objectives ensure that the project remains both academically meaningful and practically useful.

1.4 Project Scope

The scope of BuildFlow AI includes the design and implementation of a web-based planning assistant for software projects. The application supports authenticated users, project creation, multi-stage AI interaction, blueprint management, document generation, and task prompt generation. The main scope covers planning and pre-development support rather than source code generation for complete products.

The in-scope functionalities include:

1. User registration and login through Firebase authentication.
2. Storage of project data and settings in Firebase Firestore.
3. Stage-wise AI conversation for requirement elicitation.
4. Structured blueprint generation and persistence.
5. Automated generation of BRD, SRS, roadmap, and TODO outputs.
6. Creation of categorized prompts for software development activities.
7. Dashboard and workspace management for multiple projects.
8. Export support for markdown and PDF-friendly output.

The out-of-scope areas include:

1. Full automated software development from idea to deployed application.
2. Real-time multi-user collaboration in the same workspace.
3. Advanced role-based project approval workflows.
4. Automated version control integration with Git hosting platforms.
5. Native mobile application deployment.

Thus, the scope is intentionally focused on the planning and documentation layer of software engineering. This focus makes the system feasible for academic development while still solving a valuable problem.

CHAPTER 2
LITERATURE SURVEY

The literature survey for BuildFlow AI examines both traditional project planning methods and modern AI-based approaches to understand their limitations and improvements. Traditional planning relies on manual processes such as meetings, notes, spreadsheets, and document preparation. While flexible, this approach depends heavily on user experience and often results in incomplete or inconsistent documentation, especially for beginners.

Modern AI tools like ChatGPT assist in idea generation and content creation. However, they are general-purpose systems that lack structured workflows, project-specific memory, and stage-based guidance, making them less effective for systematic project planning.

BuildFlow AI bridges this gap by combining AI capabilities with a structured workflow. It organizes planning into stages, maintains project context, and generates consistent, project-specific outputs, improving the overall planning process.

2.1 Existing System

The existing system landscape may be categorized into two major groups: manual project planning methods and generic AI assistance tools.

Manual planning methods include handwritten notes, spreadsheets, presentation-based idea discussions, and document templates maintained in word processors. In these systems, users collect data themselves and then manually transform it into formal documents. The workflow is disconnected, repetitive, and often non-standardized.

Generic AI tools, including publicly available conversational systems such as ChatGPT, provide users with the ability to ask questions like "generate an SRS for my project" or "suggest features for a planning assistant." These tools can produce useful drafts and accelerate brainstorming. They can also summarize concepts, propose technology stacks, and refine wording. However, they usually operate as open-ended conversational systems rather than as complete project-planning environments.

Some productivity tools and project management platforms also offer partial assistance. For example, note-taking applications provide templates, and task managers help list work items. Yet these systems usually do not integrate contextual AI-driven requirement elicitation, document generation, and blueprint persistence in one academic workflow.

2.2 Limitations of Existing System

The limitations of existing systems are significant when viewed from the perspective of a student or small-team project planning workflow.

First, manual planning systems depend heavily on documentation effort. They require users to repeatedly rewrite the same information across notes, summaries, requirement documents, and task lists. This duplication increases time cost and introduces inconsistencies.

Second, manual methods do not actively guide the user. If a team forgets to define target users, technical constraints, or success criteria, the system provides no feedback. The planning quality therefore depends on the experience of the team.

Third, generic AI tools provide answers without enforcing a structured progression. A user may ask for an SRS before the problem definition is clear. The tool may still generate a plausible output, but the content may not align with actual project needs. This creates a false sense of completeness.

Fourth, generic AI tools do not naturally maintain a project blueprint in a dedicated application state. Users must manually restate context, copy and paste previous outputs, or manage their own notes externally. This weakens continuity.

Fifth, many generic AI tools are not tailored to academic deliverables. While they can produce content, they may omit formal sections, generate inconsistent terminology, or fail to maintain the structure expected in institutional project reports.

Sixth, conventional task management tools can store tasks but cannot derive those tasks intelligently from requirement elicitation conversations. They are organizational tools, not planning intelligence systems.

Finally, existing systems rarely connect requirement collection, document generation, and prompt generation for implementation in a single flow. Users move between multiple interfaces and lose efficiency.

These limitations establish the need for a specialized system that is not merely conversational, but workflow-aware, context-retentive, and output-oriented.

2.3 Proposed System

The proposed system is BuildFlow AI, an AI-based project planning assistant that combines structured interaction, persistent project memory, and document generation in a unified web application. The system is designed specifically for early-stage software project planning and documentation.

The core idea of the proposed system is to replace fragmented planning with a guided six-stage workflow. The first five stages collect essential planning information: idea and goal, target users and use case, core features, UI and UX preferences, and technology stack and integrations. The sixth stage focuses on output generation. At each stage, the AI assistant asks relevant questions, interprets user responses, and produces a concise summary that is stored in the project blueprint.

The proposed system offers several improvements over existing approaches:

1. Structured planning flow instead of unbounded conversation.
2. Persistent blueprint memory instead of temporary chat context.
3. Direct document generation from stored project summaries.
4. Multi-project dashboard with user-specific storage.
5. Prompt packs for downstream development activities.
6. Academic suitability through formal output generation.

Technologically, the system uses React and Vite for the frontend, enabling a fast single-page application experience. Firebase provides user authentication and Firestore-based document storage. Node.js and Express form the backend layer responsible for secure API calls, request validation, and AI orchestration. OpenRouter serves as the AI gateway, allowing model invocation through a standardized API. JSON-based structured responses are used in the chat flow to ensure that outputs can be parsed and mapped to specific blueprint stages.

The proposed system is therefore not a replacement for human planning judgment. Rather, it is a productivity and consistency amplifier that helps teams move from vague ideas to structured engineering artifacts with lower effort and greater discipline.

CHAPTER 3
SYSTEM ANALYSIS & REQUIREMENTS

3.1 Software Requirements

System analysis for BuildFlow AI focuses on identifying the technical environment, operational needs, and feasibility of implementing the proposed solution. Because the project combines web application development, cloud storage, and AI integration, its requirements must support both interactive user experience and reliable backend orchestration.

The system is developed as a browser-based platform, which allows users to access the application without specialized installation. The frontend handles presentation logic, navigation, stage-wise chat interaction, and output visualization. The backend manages AI communication and validation. Firebase provides cloud persistence and user session management. Together, these technologies support a distributed yet manageable architecture suitable for academic project deployment.

S. No.	Software Component	Purpose in BuildFlow AI	Version / Type
1	Operating System	Development and execution environment	Windows 10/11 or equivalent
2	Visual Studio Code	Source code development and debugging	Latest stable
3	Node.js	Runtime for backend and frontend tooling	Node.js 18 or above
4	npm	Package and dependency management	Bundled with Node.js
5	React	Frontend UI development	React 19
6	Vite	Frontend development server and build tool	Vite 8
7	Firebase	Authentication and Firestore database	Cloud service
8	Express.js	Backend API framework	Express 5
9	Axios / Fetch	HTTP communication for AI and app requests	Latest compatible
10	OpenRouter API	AI model access and response generation	API-based
11	Web Browser	End-user access to application	Chrome / Edge / Firefox
12	Git	Version control and collaboration	Latest stable

Table 3.1 Software Requirements

3.2 Hardware Requirements

S. No.	Hardware Resource	Minimum Requirement	Recommended Requirement
1	Processor	Dual-core CPU	Intel i5 / Ryzen 5 or above
2	RAM	4 GB	8 GB or above
3	Storage	2 GB free space	SSD with 10 GB free space
4	Network	Internet connection	Stable broadband connection
5	Display	1366 x 768 resolution	Full HD display
6	Input Devices	Keyboard and mouse	Standard laptop/desktop setup

Table 3.2 Hardware Requirements

Because the application is web-based and cloud-supported, the hardware burden on the user side is relatively low. However, reliable internet connectivity is essential due to Firebase authentication, Firestore access, and AI API communication.

3.3 Feasibility Study

The feasibility study evaluates whether the proposed system is practical from technical, economic, and operational perspectives.

Feasibility Type	Assessment	Observation
Technical Feasibility	High	The required technologies are mature, well-documented, and compatible. The team possesses skills in frontend, backend, and AI integration.
Economic Feasibility	Moderate to High	Development tools are largely free. Cloud services and AI API usage may introduce small operational costs, but these remain manageable for academic scale.
Operational Feasibility	High	Users can operate the system through familiar browser interactions. The stage-wise flow reduces training requirements and supports practical adoption.

Table 3.3 Feasibility Analysis

Technical Feasibility: The project is technically feasible because it uses widely adopted technologies with strong community and documentation support. React, Express, and Firebase are proven technologies for building modern web applications. AI integration through OpenRouter is possible via standard REST communication. The development team structure also aligns with project needs, with identified roles in development, documentation, and research.

Economic Feasibility: The system is cost-effective for academic use. Development tools such as Node.js, React, and Visual Studio Code are free. Firebase offers a free tier suitable for small-scale testing and demonstration. OpenRouter-based AI access may incur usage costs depending on the selected model and traffic volume, but these costs remain low for project evaluation and limited demonstration use.

Operational Feasibility: The system is easy to operate because users interact with it through familiar web forms, buttons, tabs, and chat interfaces. The workflow is designed to minimize cognitive overload by focusing on one planning stage at a time. This increases adoption potential among students and beginner teams who may otherwise be intimidated by formal documentation tasks.

From the combined analysis, BuildFlow AI is feasible and suitable for academic implementation within the given project timeline and available resources.

CHAPTER 4

SYSTEM DESIGN

System design translates the problem solution into a structured technical blueprint. The design of BuildFlow AI is centered around modularity, clarity of control flow, and clean separation between presentation, persistence, and AI orchestration. The application is divided into a client-side interface, a backend service layer, a cloud persistence layer, and an external AI service.

4.1 System Architecture

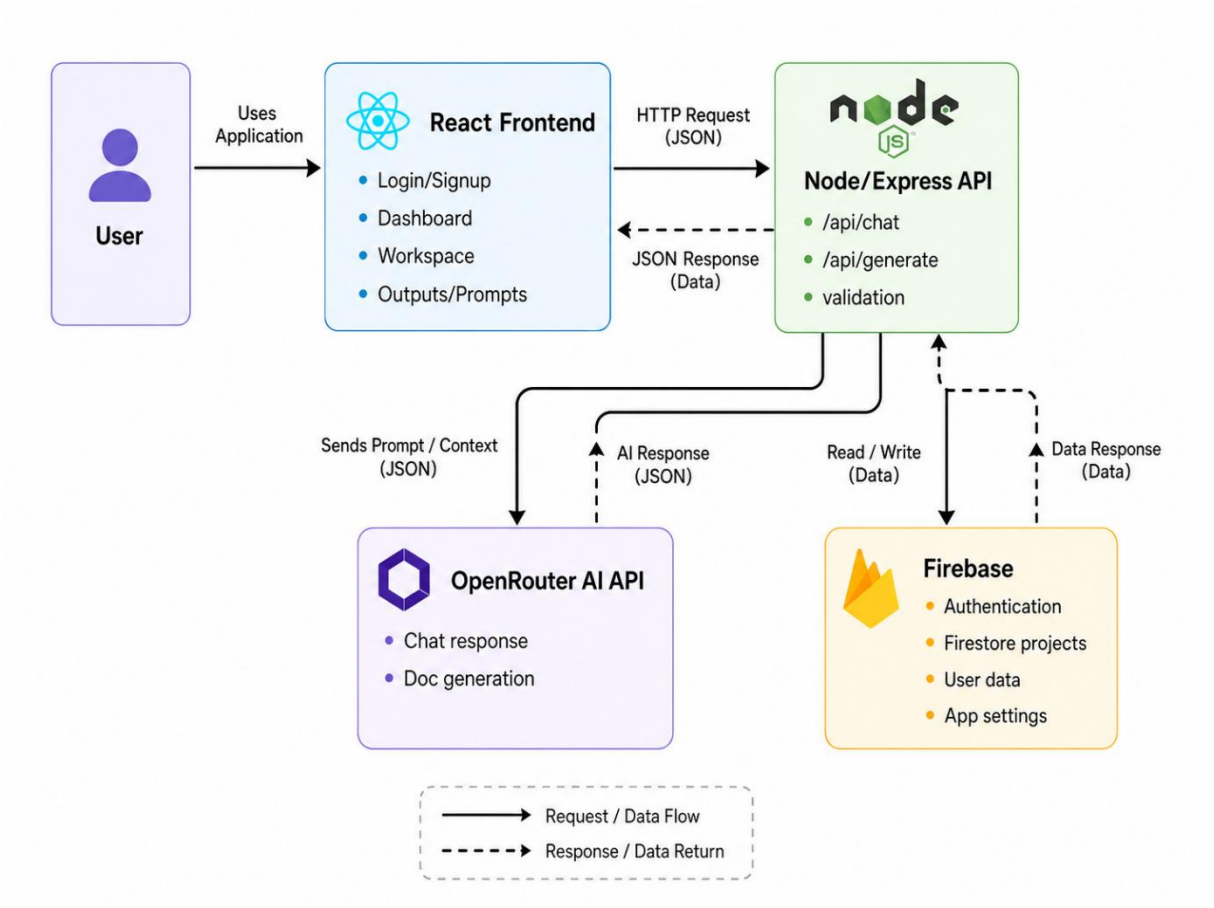

Figure no 4.1 System Architecture

Architecture Explanation:

1. The user interacts with the React frontend through a browser.
2. The frontend provides navigation, forms, chat panels, blueprint display, output tabs, and prompt panels.

3. When the user submits planning responses, the frontend sends stage data and recent messages to the Express backend.
4. The backend enriches the request with a system prompt, user role, stage information, and blueprint memory.
5. The backend calls the AI model through OpenRouter.
6. The AI response is parsed and returned as structured JSON for the chat workflow or markdown text for generated documents.
7. Firebase authentication manages login state, while Firestore stores projects, outputs, prompts, and user settings.

This architecture was chosen because it separates concerns effectively. Sensitive AI API keys remain on the backend rather than the browser, while project data persistence is handled by a scalable managed service.

4.2 Data Flow Diagram (DFD)

Level 0 DFD

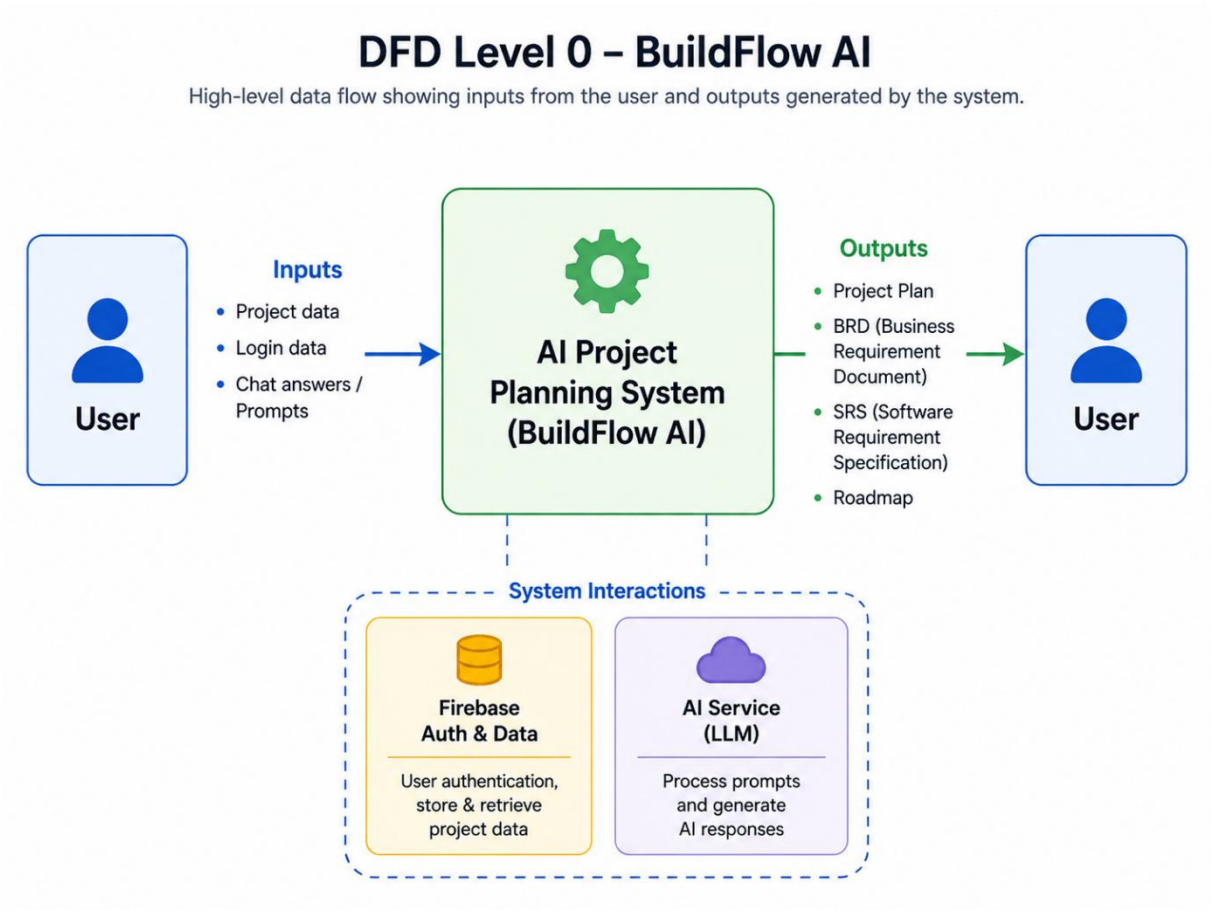

Figure no 4.2 Data Flow Diagram (Level 0)

Explanation:

1. **User (Left Side – Input)**

   The user interacts with the system by providing inputs such as:

   - Project details
   - Login information
   - Chat prompts or questions

2. **AI Project Planning System (Center Process)**

   All the user inputs go into the main system called **BuildFlow AI**.
   This system processes the data and uses AI to understand the requirements and generate useful outputs.

3. **System Processing**

   Inside the system:

   - It analyzes the user input
   - Applies AI logic to generate project-related content
   - Prepares structured outputs based on the given data

4. **Outputs (Result)**

   After processing, the system sends results back to the user.
   The outputs include:

   - Project Plan
   - BRD (Business Requirement Document)
   - SRS (Software Requirement Specification)
   - Roadmap

5. **User (Output Receiver)**

   The same user receives the generated outputs and can view them in the application.

Level 1 DFD

1. Authentication Module (1.0)

The user provides login details such as email and password.
These credentials are sent to Firebase Authentication, which verifies them and returns a success or failure response.

2. Project Management Module (2.0)

The user can create, update, or delete project data.
This data is stored and retrieved from Firestore Database, ensuring persistent project management.

3. Stage Chat Module (3.0)

The user interacts with the system through chat inputs.
This module maintains:

- Recent messages
- Blueprint memory
- Stage metadata

The data is sent to the Express Backend, which forwards it to the OpenRouter AI (LLM).
The AI processes the input and returns a response, which is then converted into a structured JSON format for further use.

4. Output Generation Module (4.0)

Based on the processed data and AI responses, the system generates structured outputs such as:

- BRD (Business Requirement Document)
- SRS (Software Requirement Specification)
- Roadmap
- TODO lists

5. Prompt Generation Module (5.0)

The system also generates role-based prompts for development tasks, including:

- Frontend (UI)
- Backend
- Deployment

These prompts help users in building and implementing their projects effectively.

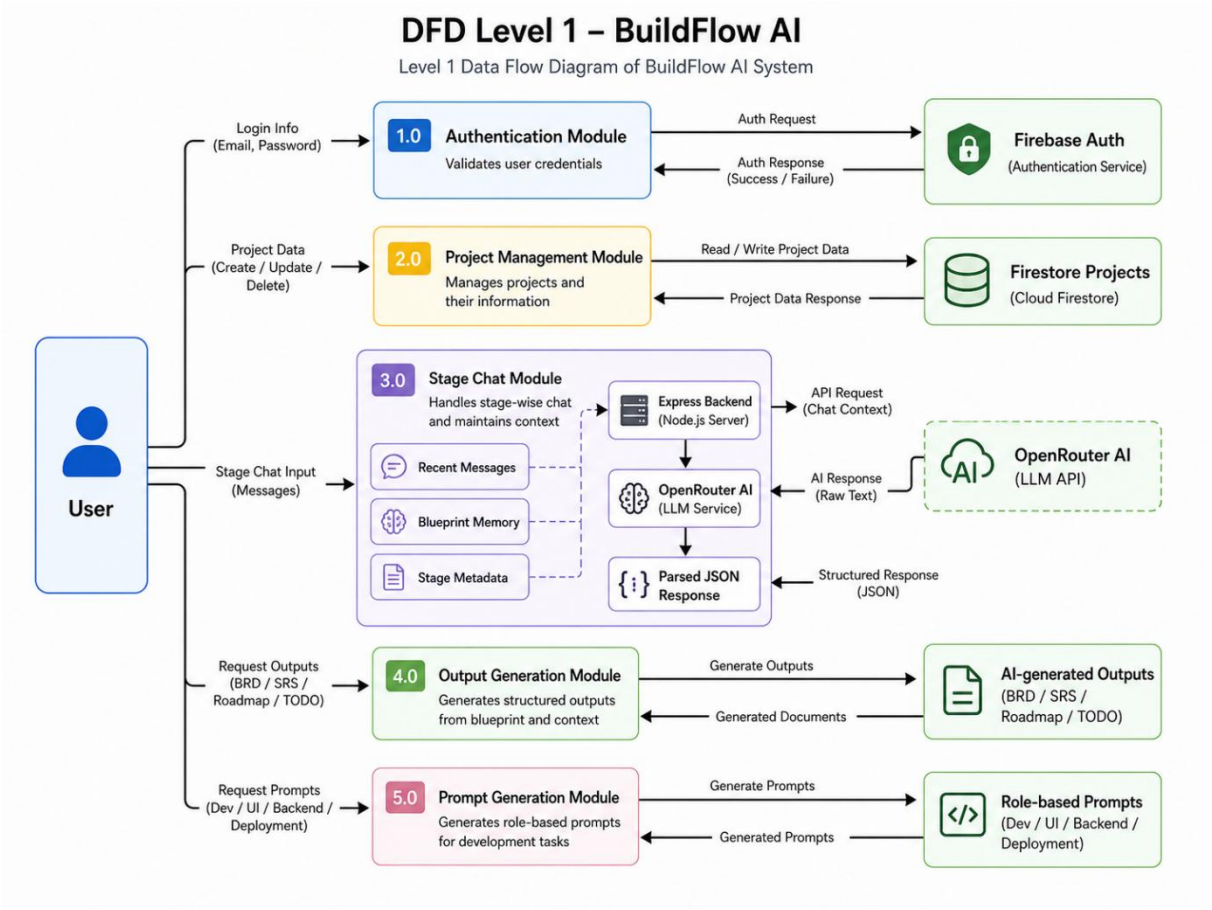

Figure no 4.3 Data Flow Diagram (Level 1)

4.3 UML Diagrams

Use Case Diagram

The Use Case Diagram represents the interaction between the User and the BuildFlow AI system. It shows the different functionalities that the user can perform within the system.

Actors:

- **User** – The primary actor who interacts with the system.

Use Cases:

- Sign Up
- Log In
- Create Project
- View Dashboard
- Open Workspace
- Answer Stage Questions
- View Blueprint
- Generate BRD
- Generate SRS
- Generate Roadmap
- Generate TODO
- Generate Task Prompts
- Export Markdown
- Export PDF
- Manage Settings

- **Manage Settings**

The user updates account settings and preferences.

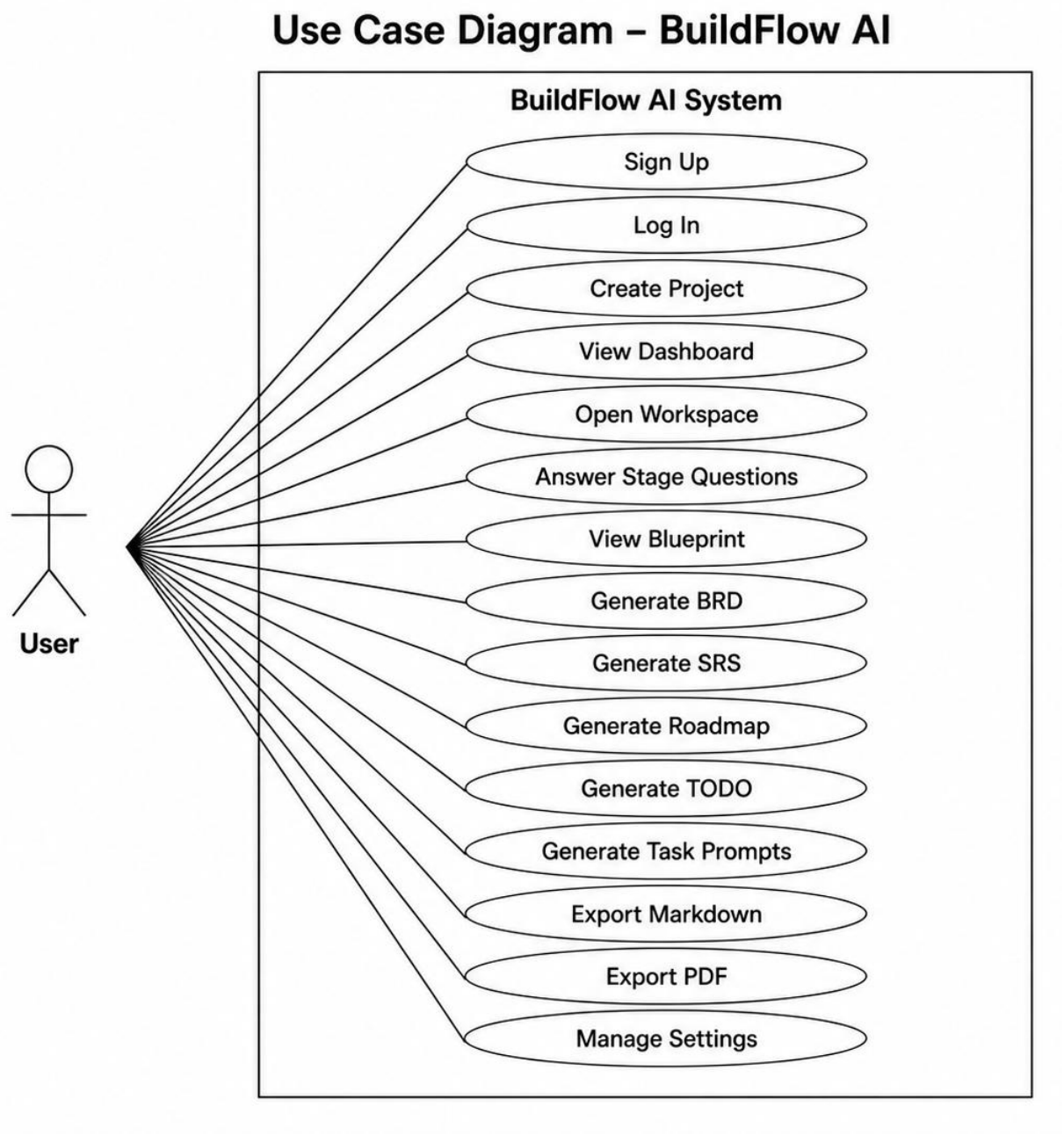

Figure no 4.4 Use Case Diagram

Class Diagram

Main Classes:

- **User** stores basic details like id, email, and name. Each user has one **Settings** and can create multiple **Projects**.
- **Settings** stores user preferences and role.
- **Project** is the main part of the system. It contains all project details and is connected to:
    - **Blueprint** (project stages)
    - **Outputs** (BRD, SRS, Roadmap, TODO)
    - **PromptPack** (development prompts)

- **Blueprint** stores step-by-step stages of the project.
- **Outputs** contains all generated documents.
- **PromptPack** stores prompts for frontend, backend, UI, and deployment.
- **ChatService** handles chat between user and AI.
- **AIService** processes AI requests and connects to the **OpenRouter API** to generate

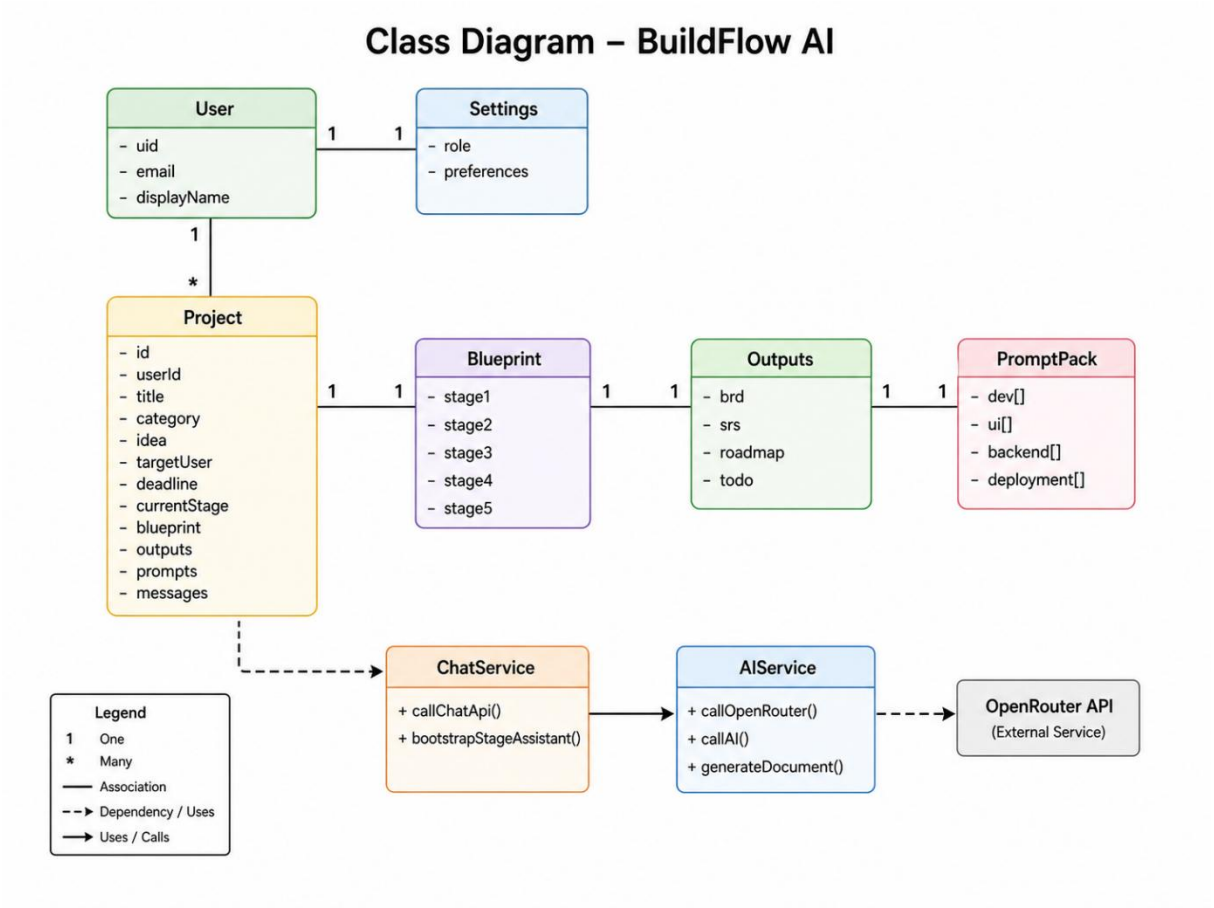

Figure no 4.5 Class Diagram

Activity Diagram

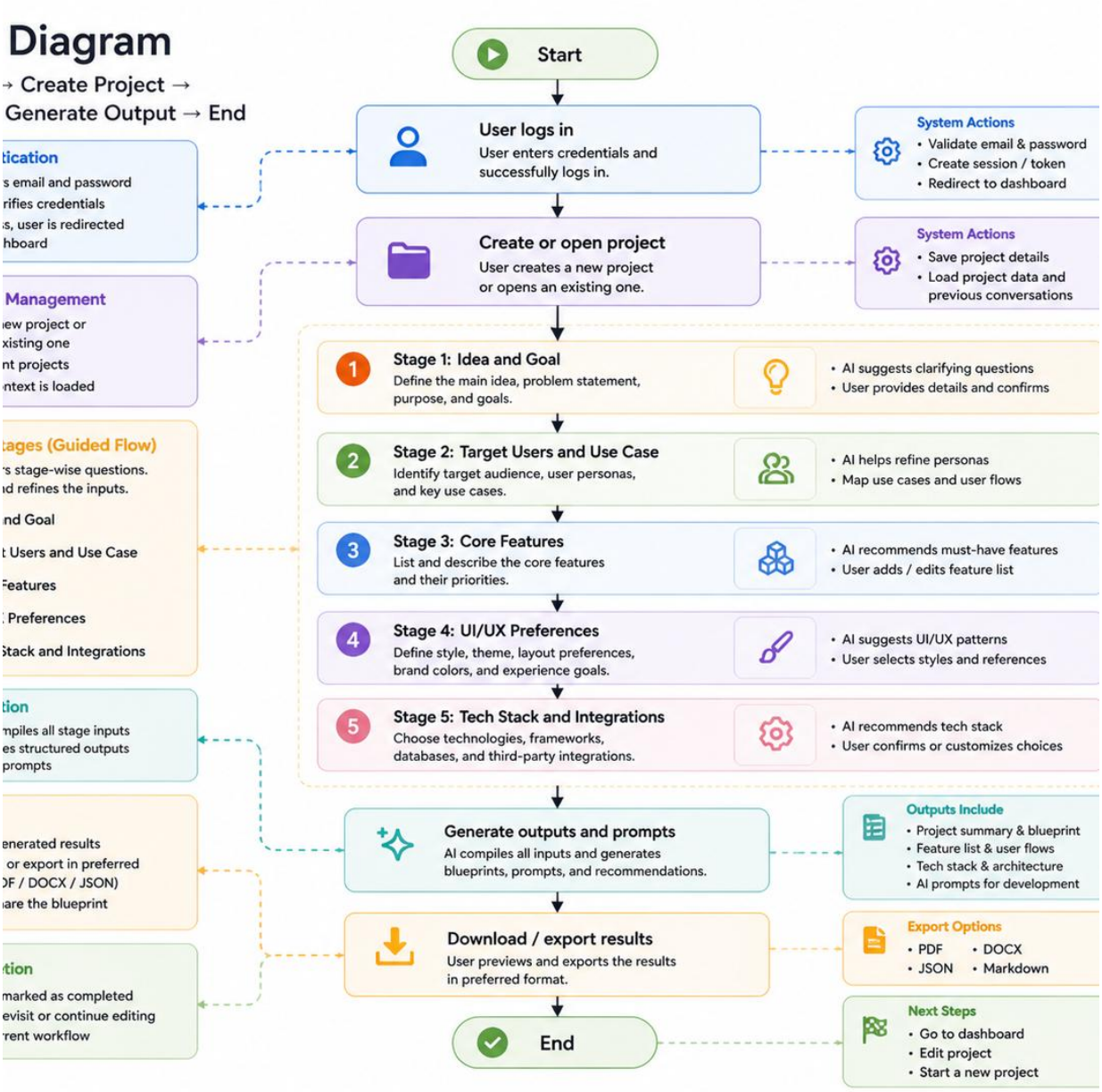

Figure no 4.6 Activity Diagram

Sequence Diagram

Interaction:

User → Frontend Workspace → Firestore → Backend (/api/chat) → AIService → OpenRouter API → AIService → Backend → Frontend Workspace → Firestore → UI Display

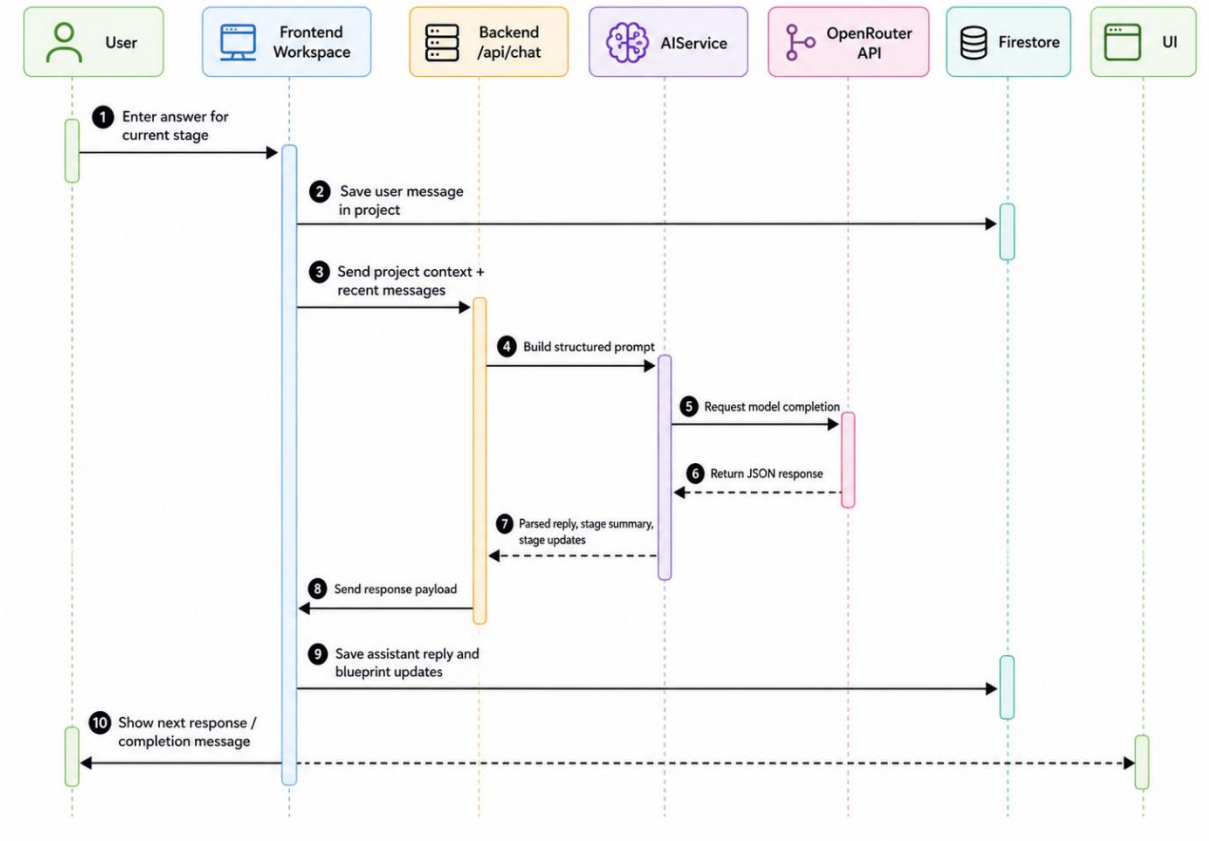

Figure no 4.7 Sequence Diagram

CHAPTER 5

IMPLEMENTATION

Implementation of BuildFlow AI involved coordinated development across the frontend, backend, cloud database, and AI integration layers. The overall implementation strategy emphasized reusability, state normalization, controlled AI behavior, and practical usability. The frontend was structured into pages, context providers, utility modules, and reusable components. The backend was kept intentionally lightweight and focused on API orchestration, request validation, and AI response handling.

5.1 Modules Description

Module	Description	Key Files / Logic
Authentication	Handles signup, login, logout, and auth state	`AuthContext.jsx`, `firebase.js`
Dashboard	Lists projects and allows workspace navigation	Dashboard page and project components
Chatbot	Conducts stage-wise AI interaction	`WorkspacePage.jsx`, `routes/chat.js`
Blueprint	Stores and displays summarized stage data	`projectStorage.js`, blueprint panel
Output Generation	Generates BRD, SRS, roadmap, TODO	`routes/generateDocs.js`, `aiService.js`, outputs panel
Settings	Stores user role and preferences	`firestoreService.js`, settings page

Table 5.1 Module Responsibility Mapping

Authentication Module:

The authentication module uses Firebase Authentication. In the React context layer, the application subscribes to authentication state changes using `onAuthStateChanged`. The signup flow creates a user with email and password and updates the user profile name. The login flow uses Firebase sign-in methods, and logout ends the session. This module ensures that projects are tied to user identities and that personal workspaces remain separated.

Dashboard Module:

The dashboard acts as the landing space after login. It retrieves projects associated with the logged-in user from Firestore and presents them in a navigable interface. Each project contains metadata such as title, category, deadline, current stage, blueprint, outputs, prompts, and messages. The dashboard gives the user quick access to continue planning or create a new project.

Chatbot Module:

This is the core planning engine of the application. The workspace retrieves project data, determines the current stage, and allows the user to send answers. The frontend constructs a request payload containing project metadata, recent messages, blueprint summaries, and user role. The backend route `/api/chat` accepts this payload, builds a structured prompt, sends it to the AI service, parses the JSON response, and returns reply data.

Blueprint Module:

The blueprint module is responsible for turning conversation into stored planning knowledge. Each of the first five stages corresponds to one blueprint field. When the AI indicates completion or returns a stage summary, the application updates the relevant blueprint field. This blueprint is then used across the system, especially in document generation and prompt generation.

Output Generation Module:

The outputs panel allows users to generate BRD, SRS, roadmap, and TODO documents. The backend route `/api/generate-docs` validates the input project and output type, ensures that planning has started, and asks the AI service to generate structured markdown content. The frontend stores generated content in the project outputs object and provides actions to copy, download, or export the content.

Settings Module:

The settings module stores additional user-specific data such as role. This role is included in backend AI requests so that responses can align more closely with the type of user interacting with the system, for example a student developer, founder, or planner.

5.2 Algorithms / Techniques Used

Technique	Purpose	Application in BuildFlow AI
Prompt Engineering	Constrain AI behavior and improve relevance	System prompt and document prompt construction
JSON Extraction	Convert AI output into machine-usable data	Structured parsing in chat route
Stage-based Flow Control	Enforce planning order	Current stage logic and completion checks
Context Window Trimming	Limit noise and reduce token load	Sending only recent messages
State Normalization	Preserve project integrity	`normalizeProject()` in storage utility
Heuristic Completion Detection	Determine when a stage is complete	Summary presence and stage completion flags

Table 5.2 Algorithms and Techniques Used

Prompt Engineering:

Prompt engineering is central to BuildFlow AI. The backend defines a strict system prompt requiring valid JSON output for stage chat responses. This prompt specifies the expected fields: `reply`, `stage_complete`, `stage_summary`, and `stage_updates`. By constraining the format, the system reduces ambiguity and ensures that responses can be parsed programmatically.

JSON Extraction Technique:

The backend uses a safe parsing strategy to interpret AI responses. It first attempts direct JSON parsing. If that fails, it searches for the first JSON-like object within the returned text and attempts a second parse. This fallback improves robustness against occasional model formatting drift.

Stage-based Flow Control:

The application restricts planning to defined stages. The current stage is clamped within valid bounds, and previous blueprint data is selectively included as memory only up to earlier stages. This avoids contaminating the current stage with future-stage assumptions and preserves logical progression.

Context Window Trimming:

Instead of sending the entire conversation history, the frontend forwards only the last six messages for the current stage. This design reduces unnecessary token usage and keeps the AI focused on recent context while still benefiting from blueprint memory.

State Normalization:

Projects in Firestore are normalized to include consistent default shapes for blueprint, outputs, prompts, and messages. This prevents null-state errors and allows all components to operate safely even when some fields were not initially present.

Heuristic Completion Detection:

Stage completion is determined using both explicit AI flags and the presence of stored blueprint summaries. This dual strategy improves user experience because a stage can be considered complete if the system has already captured sufficient structured information, even if the chat wording varies.

5.3 Key Code Snippets

The following implementation excerpts illustrate the working principles of the system.

Snippet 1: Backend route registration

```
app.get('/api/health', (_req, res) => {
  res.json({ ok: true })
})

app.use('/api/chat', chatRouter)
app.use('/api/generate-docs', generateDocsRouter)
```

This snippet shows the backend exposing a health endpoint and separating chat and document generation into dedicated routes.

Snippet 2: Limiting memory to relevant prior blueprint stages

```
const filteredMemory = stageKeys.reduce((acc, key, idx) => {
  const value = mergedBlueprint[key]
  if (idx < stageLimit && typeof value === 'string' &&
value.trim()) {
    acc[key] = value.trim()
  }
  return acc
}, {})
```

This logic ensures that only completed earlier stages are included as context, which keeps the AI grounded in valid planning history.

Snippet 3: Safe AI response parsing

```
function safeParseAIResponse(raw) {
  if (typeof raw !== 'string') return null
  try {
    return JSON.parse(raw)
  } catch {
    const match = raw.match(/\{[\s\S]*\}/)
    return match ? JSON.parse(match[0]) : null
  }
}
```

This snippet illustrates defensive programming for AI integration. Because model outputs can sometimes drift, the system includes a fallback extraction strategy.

Snippet 4: Project normalization

```
return {
  ...project,
  currentStage,
  blueprint: { ...defaultBlueprint, ...(project?.blueprint || {})
},
  outputs: { ...defaultOutputs, ...(project?.outputs || {}) },
  prompts: { ...defaultPrompts, ...(project?.prompts || {}) },
  messages: normalizedMessages,
}
```

This normalization pattern helps maintain consistent application state and reduces conditional complexity across UI components.

Snippet 5: Document generation request from frontend

```
const response = await fetch(`${BASE_URL}/api/generate-docs`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ type, project }),
})
```

This request connects the output generation panel to the backend AI document generation service.

Overall, the implementation demonstrates the practical integration of modern frontend architecture, backend AI orchestration, and cloud persistence into a focused planning product.

CHAPTER 6

TESTING

Testing was conducted to validate the functionality, reliability, and usability of BuildFlow AI. Since the system includes frontend interaction, backend endpoints, third-party authentication, cloud persistence, and AI-dependent behavior, testing had to cover multiple layers. The objective was not only to verify whether the system runs, but also whether it supports the intended planning workflow without failure or inconsistency.

6.1 Testing Methods

The following testing methods were applied:

1. Unit-oriented logic testing for utility behavior such as normalization and stage handling.
2. Functional testing of core modules including authentication, project creation, stage chat, output generation, and prompt generation.
3. API testing for backend routes such as `/api/health`, `/api/chat`, and `/api/generate-docs`.
4. UI testing for workspace navigation, stage progression, button actions, and generated content rendering.
5. Error-path testing for missing backend URL, missing project payload, missing planning data, and AI failure handling.

Test Area	Objective	Expected Condition
Authentication	Validate signup/login/logout flows	User session should update correctly
Dashboard	Validate project listing and selection	User projects should load correctly
Chat Flow	Validate stage-wise AI interaction	Reply, summary, and updates should be returned
Blueprint Update	Validate stage summary persistence	Relevant stage fields should be stored
API Validation	Validate bad request handling	Proper errors should be returned
Output Generation	Validate BRD/SRS/roadmap/TODO generation	Output content should be stored and viewable
Prompt Generation	Validate prompt pack creation	Category prompts should populate correctly
Export Behavior	Validate markdown/PDF actions	File download or print dialog should trigger

Table 6.1 Functional Testing Coverage

Manual black-box testing was especially useful because the system includes conversational behavior that depends on external AI responses. Functional validation therefore focused on input-output correctness, state transitions, and graceful error recovery.

6.2 Test Cases

Test Case	Input	Expected Output	Result
User Signup	Valid email, password, and name	Account created and user logged in	Pass
User Login	Registered email and password	Dashboard opens with authenticated session	Pass
Invalid Login	Incorrect password	Error message displayed	Pass
Create Project	Title, category, idea, target user	New project stored in Firestore	Pass
Open Workspace	Select existing project	Workspace loads correct project and stage	Pass
Bootstrap Chat	Open a stage with no messages	AI asks first relevant stage question	Pass
Send Stage Answer	Valid user response in current stage	AI returns reply and summary	Pass
Stage Completion	Sufficient stage details provided	Stage marked complete and continue enabled	Pass
Blueprint Persistence	Summary returned by AI	Summary saved under matching stage key	Pass
Health API	GET `/api/health`	JSON `{ ok: true }`	Pass
Invalid Chat Payload	Missing or malformed body	Error message returned gracefully	Pass
Invalid Doc Type	Unsupported type	400 response with validation message	Pass
Generate BRD	Valid project with stage1 complete	BRD markdown generated and stored	Pass
Generate SRS	Valid project with stage1 complete	SRS markdown generated and stored	Pass
Generate All Docs	Valid project and active backend	All four outputs stored	Pass
Generate Prompts	Click category generation	Prompt list populated in active category	Pass
Export Markdown	Generated document present	Download of markdown file starts	Pass
Export PDF	Generated document present	Print dialog opens for PDF save	Pass
Missing Backend URL	Frontend backend URL unset	User receives configuration error	Pass
Missing Initial Blueprint	Request docs before planning	User receives guidance to finish initial chat	Pass

Table 6.2 Detailed Test Cases

CHAPTER 7
RESULT & DISCUSSION

The results of BuildFlow AI indicate that the system successfully transforms fragmented project ideas into structured planning outputs. The stage-wise workflow proved effective in reducing ambiguity and guiding the user toward complete planning coverage. Instead of asking for a large one-time prompt, the system collects information gradually and stores it as a blueprint. This blueprint then becomes the basis for downstream outputs.

The project produced four major generated artifacts: BRD, SRS, roadmap, and TODO. In addition, it generated categorized implementation prompts for development, UI, backend, and deployment tasks. Together, these outputs demonstrate that the system can bridge the gap between ideation and engineering preparation.

Discussion of System Behavior:

The most significant result is the conversion of conversational input into reusable structured state. Unlike generic chat systems, BuildFlow AI does not leave information buried in message history alone. Stage summaries are extracted and stored directly into the blueprint, which becomes the central planning memory of the application.

Another positive result is modular output generation. Users can generate a single document type or all document types together. This makes the platform flexible for different use cases. A team may initially want only a roadmap, and later produce the full documentation set after more planning maturity.

Prompt generation is also an important outcome. Instead of ending at formal documentation, the system extends into execution support by generating categorized prompts for developers and assistants. This increases the practical value of the platform, especially when teams use AI-supported coding workflows.

The overall results also show that workflow control matters. The stage model improves the quality of collected information because each interaction is narrowed to a specific planning concern. This is especially useful for beginners who may not know what information should be documented first.

7.1 Screenshots / Output Interface

This section presents the user interface of BuildFlow AI in a logical workflow, from user entry to final output generation.

1. Landing Page Interface

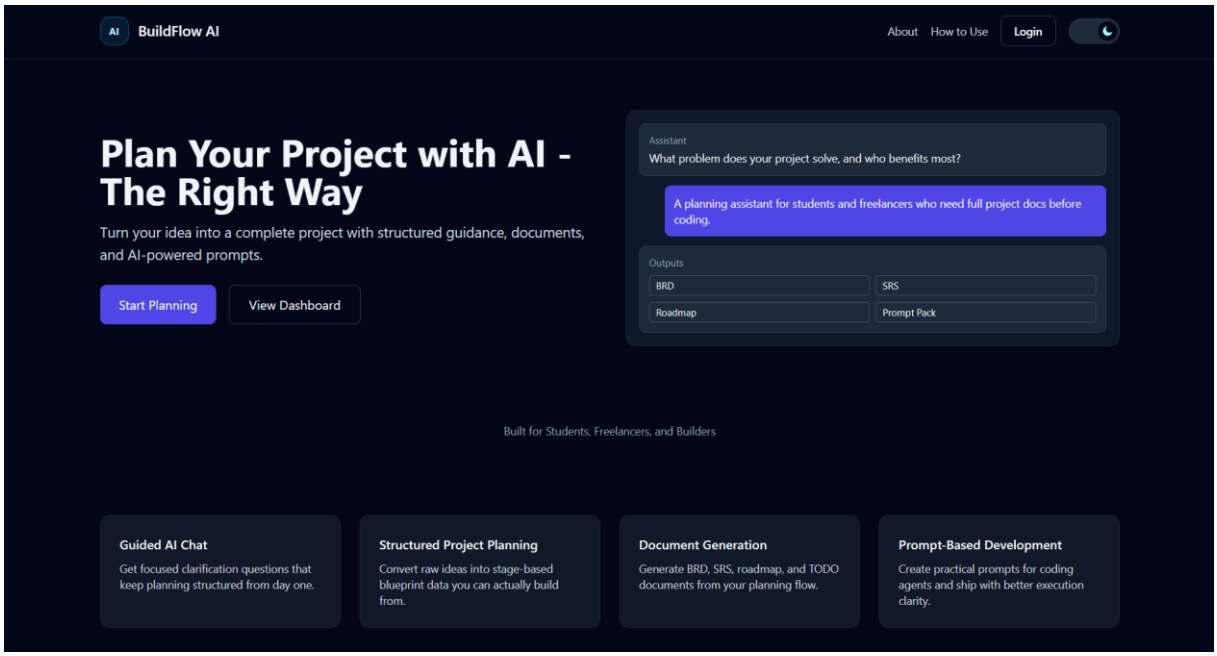

Figure 7.1 Landing Page Interface

2. User Registration Interface

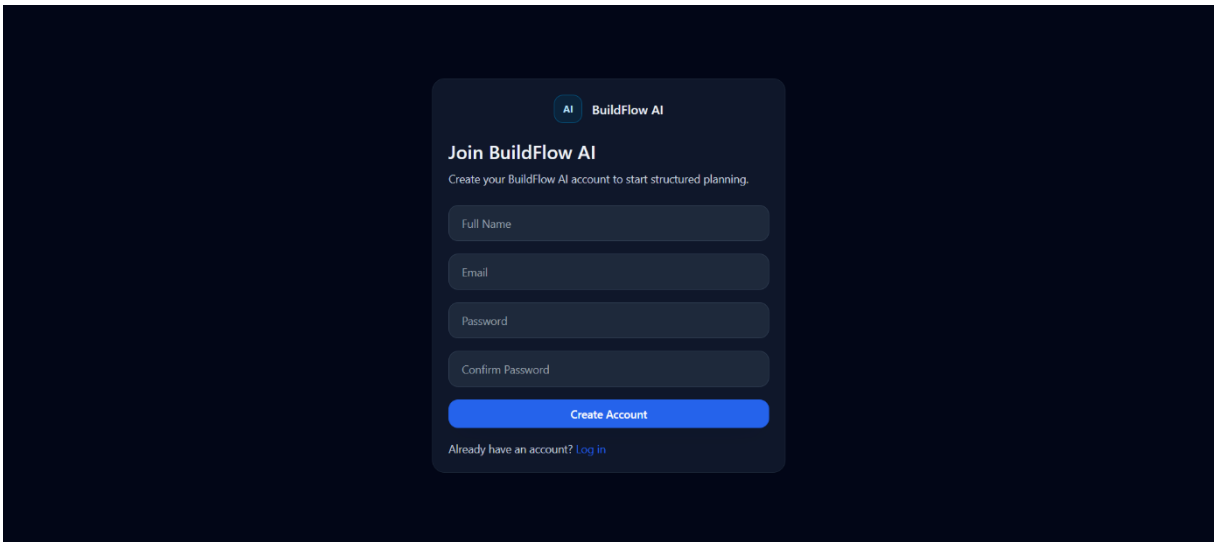

Figure no 7.2 User Registration Interface

3. User Login Interface

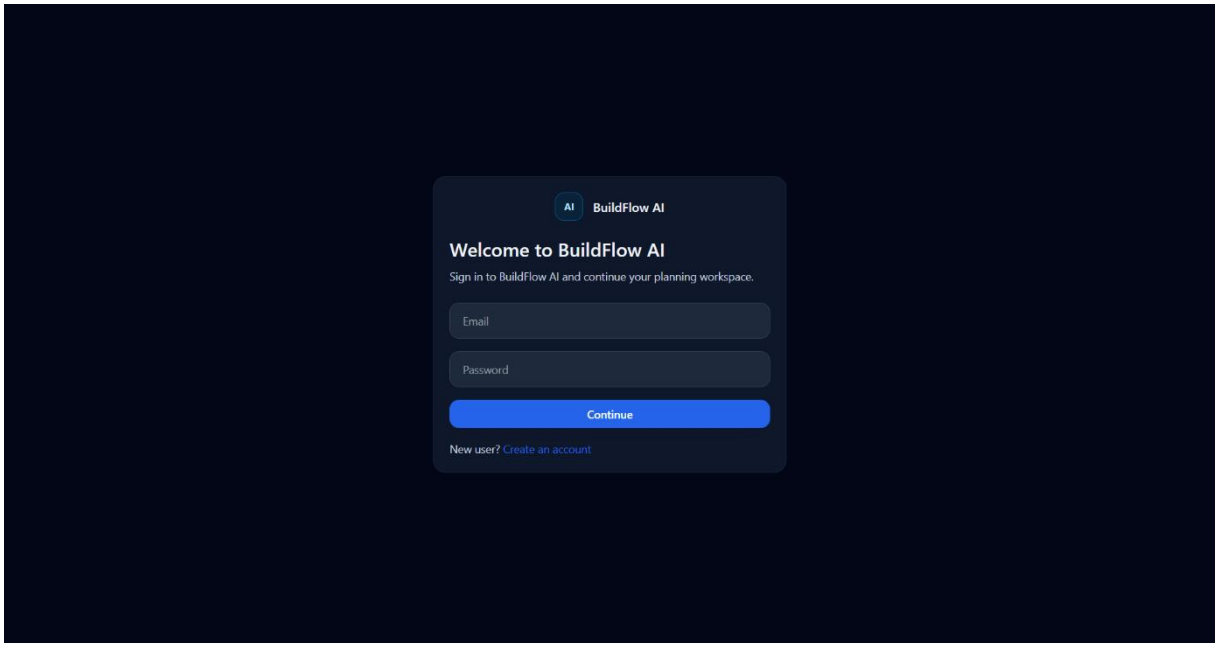

Figure no 7.3 User Login Interface

4. Dashboard Interface

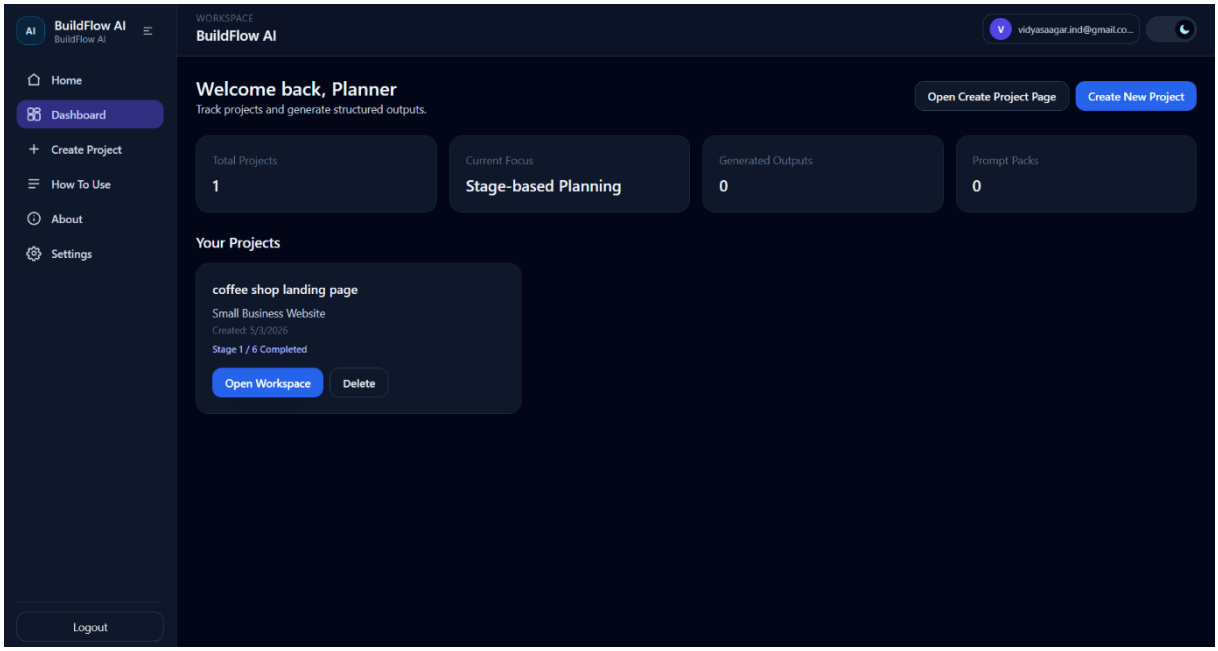

Figure no 7.4 Dashboard Interface

5. Create Project Interface

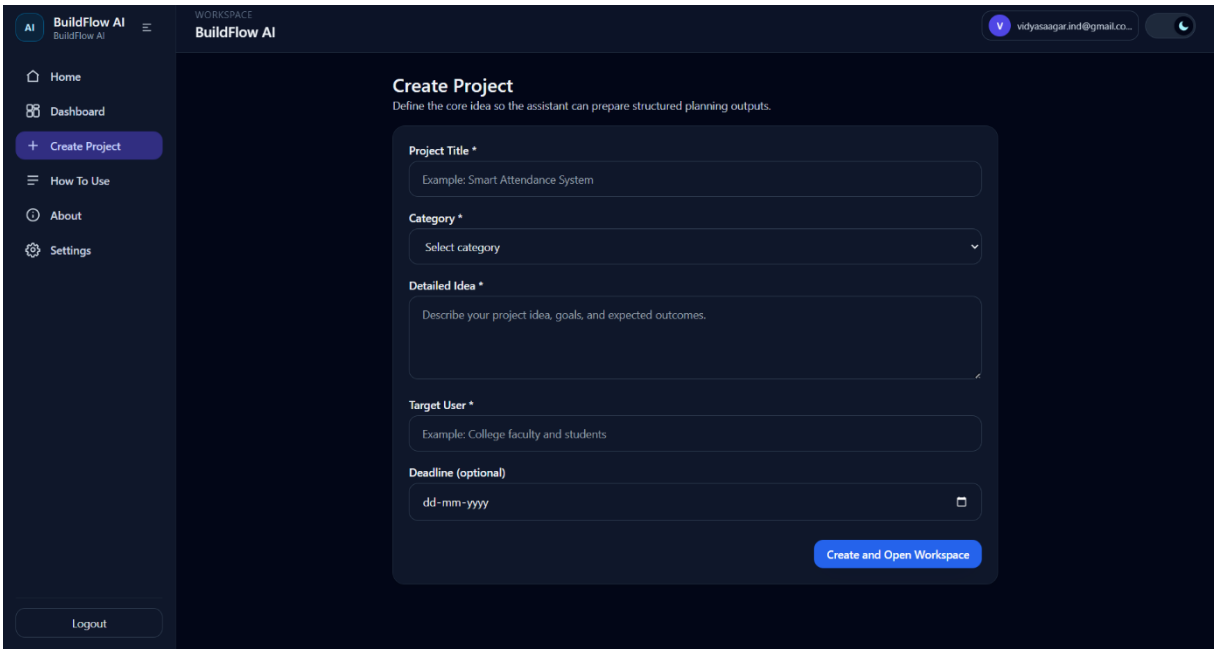

Figure no 7.5 Create Project Interface

6. Workspace – Chatbot Interaction Interface

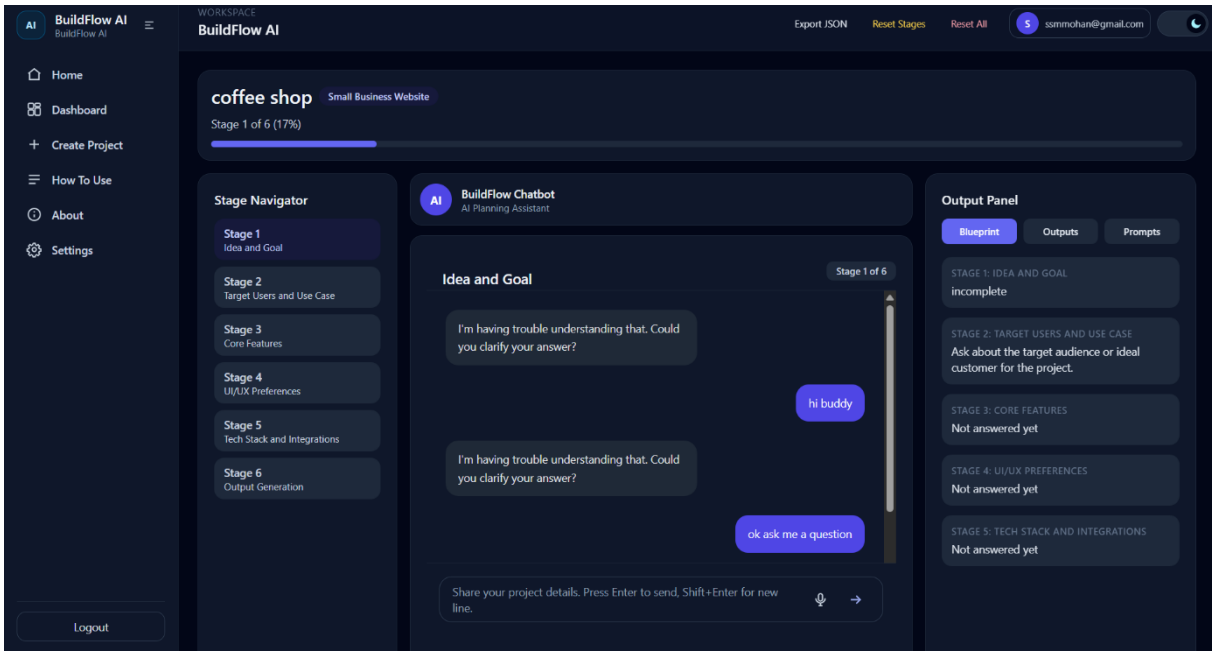

Figure no 7.6 Workspace – Chatbot Interaction

7. Stage-Based Planning Interface

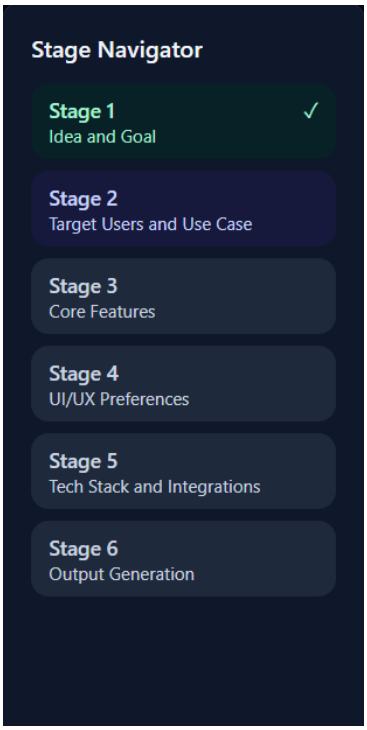

Figure no 7.7 Stage-Based Planning Interface

8. Blueprint Panel Interface

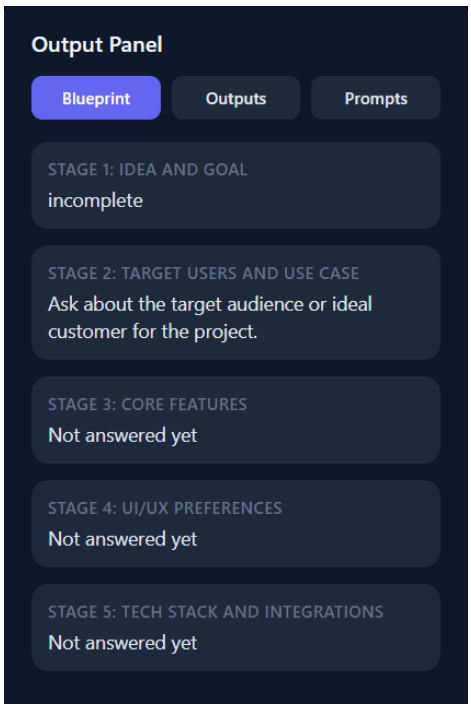

Figure no 7.8 Blueprint Panel Interface

9. Output Panel Interface

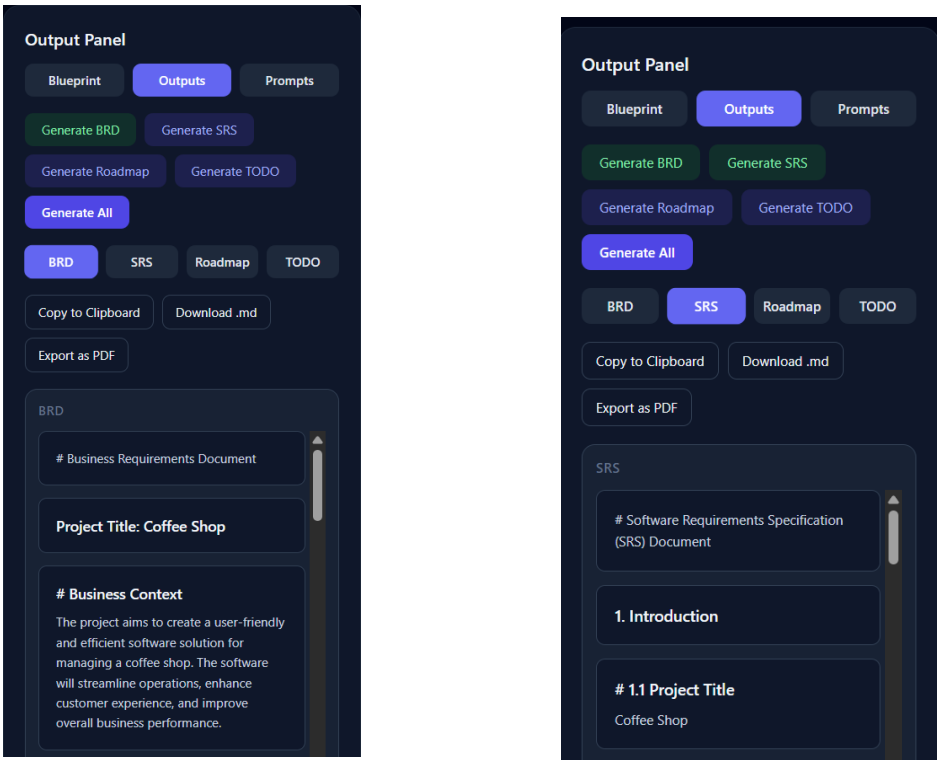

Figure no 7.9 Output Panel Interface

10. Prompt Generation Interface

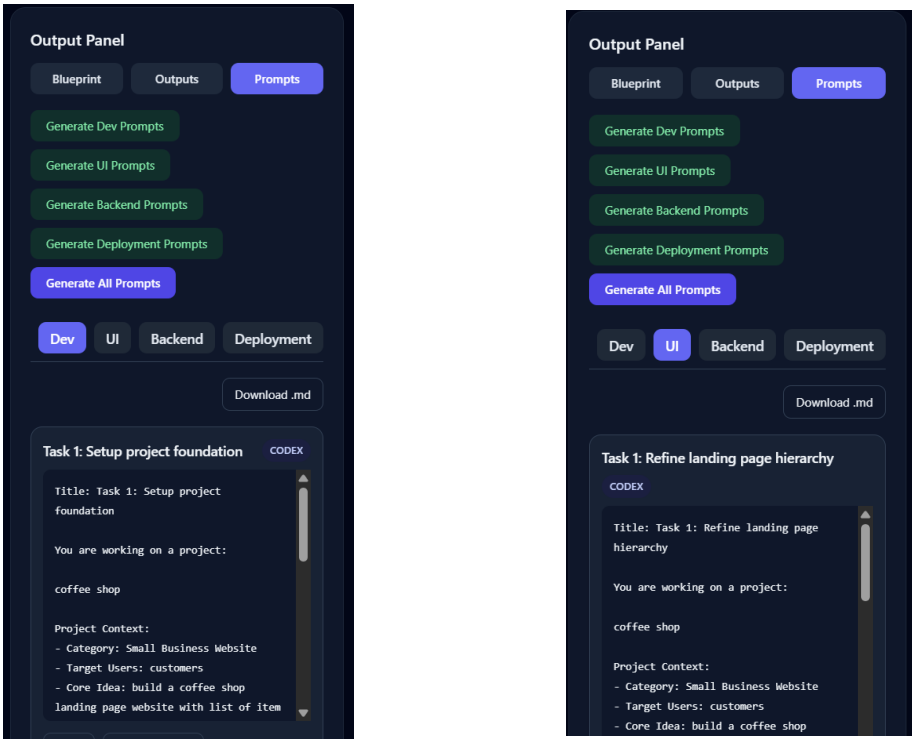

Figure no 7.10 Prompt Generation Interface

11. Settings Interface

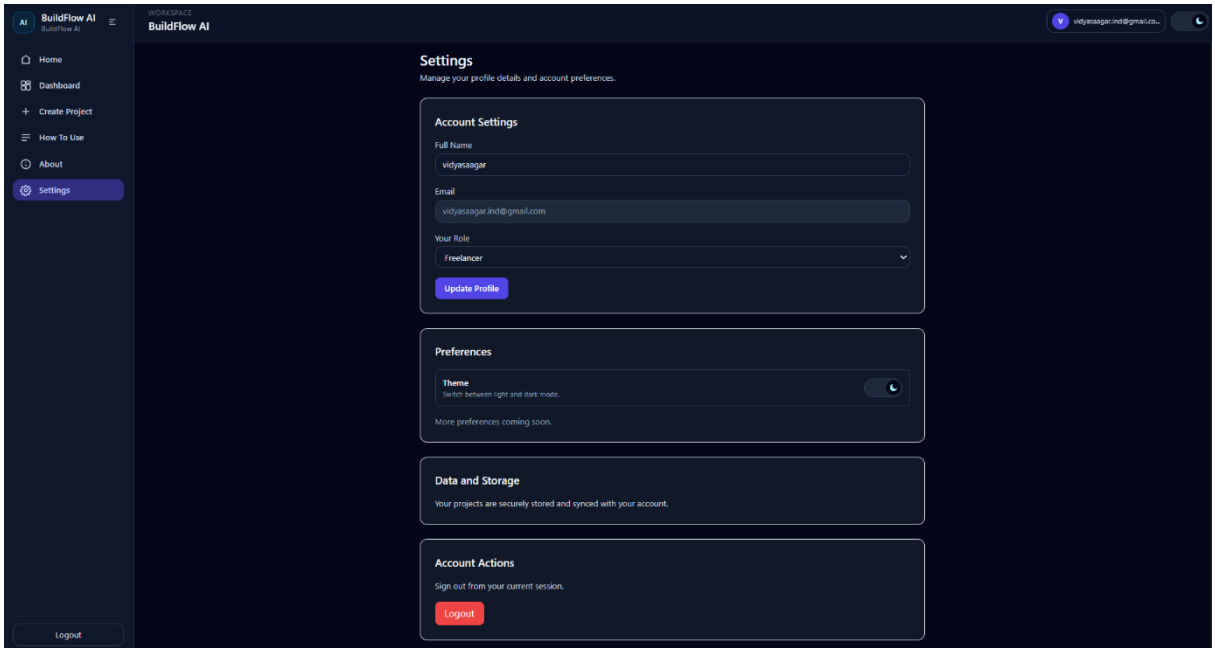

Figure no 7.11 Settings Interface

12.Theme Customization Interface (Light/Dark Mode)

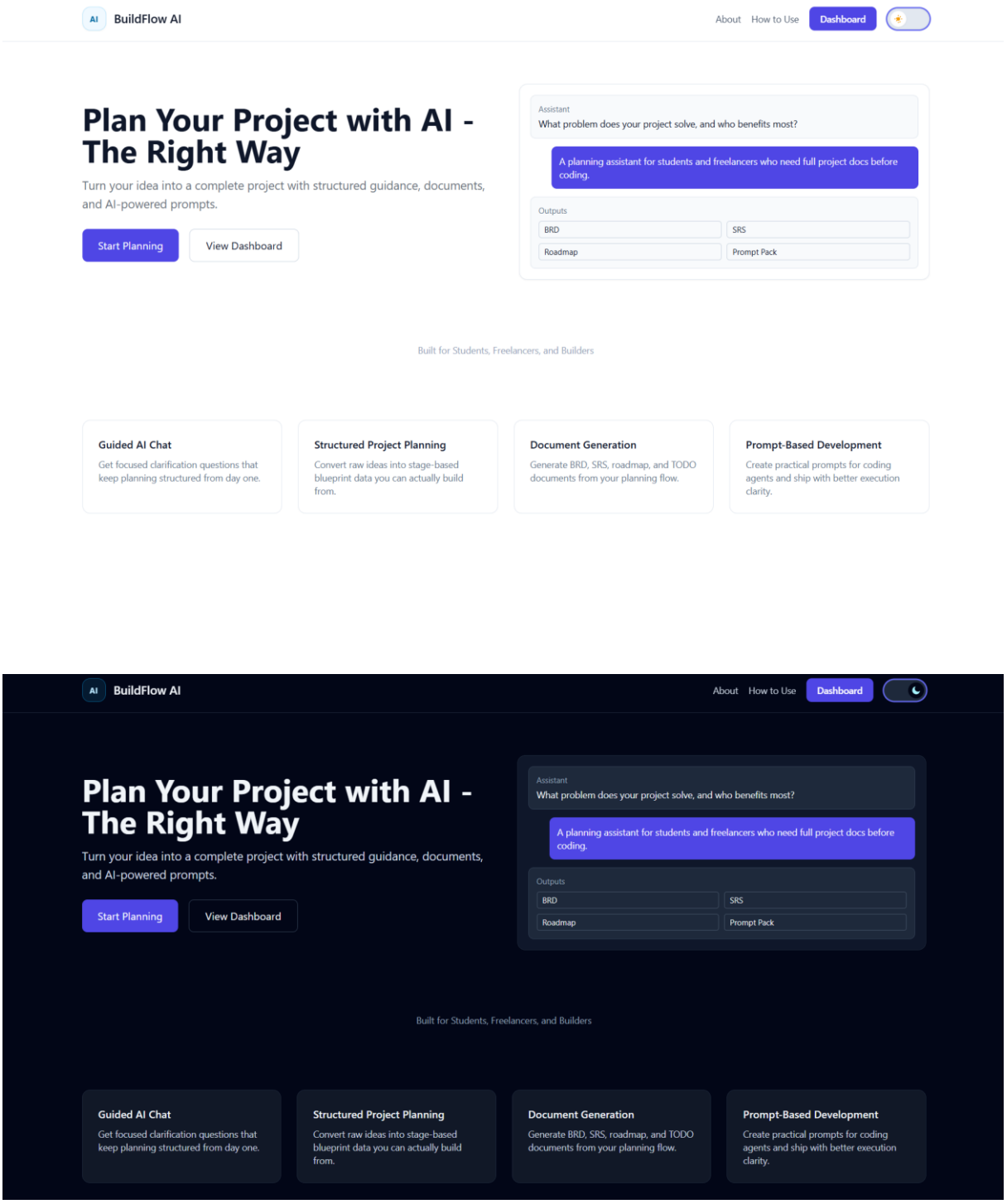

Figure no 7.12 Light and Dark Theme Interface

CHAPTER 8

CONCLUSION & FUTURE ENHANCEMENT

8.1 CONCLUSION

BuildFlow AI successfully demonstrates that artificial intelligence can be integrated into software project planning in a structured, academically useful, and practically relevant manner. The project addresses a clear problem: early-stage software planning is often inefficient, inconsistent, and difficult for student teams to formalize. By introducing a guided multi-stage workflow, persistent blueprint memory, and automated output generation, the system reduces planning friction and transforms raw ideas into organized engineering artifacts.

The project achieved its core objectives. It provides user authentication, project storage, stage-wise AI interaction, blueprint management, document generation, prompt generation, and export support. The system architecture is modular, the implementation is based on modern and widely used web technologies, and the outputs align well with academic documentation needs. Most importantly, the system does not rely on open-ended chat alone. It constrains the AI within a planning process, making its outputs more consistent and actionable.

From an academic perspective, BuildFlow AI is a meaningful contribution because it blends software engineering methodology with applied AI. It demonstrates requirement elicitation, design thinking, system architecture, implementation, testing, and evaluation within one integrated project. From a practical perspective, it offers a reusable platform that could benefit students, early-stage founders, freelancers, and small teams who need faster project planning support.

8.1 FUTURE ENHANCEMENT

1. Improved AI Models and Prompt Strategies

Future versions can use stronger models, adaptive prompting, and more advanced response validation. This could improve summarization accuracy, domain adaptation, and document depth.

2. Deployment Automation

The system can be extended to generate CI/CD checklists, deployment templates, infrastructure recommendations, and release pipelines directly from the collected blueprint.

3. Collaboration Features

Multi-user shared workspaces, commenting, approval flows, and change tracking can make the platform suitable for team collaboration rather than individual planning only.

4. Versioned Blueprint Editing

Future enhancement may include blueprint revision history, comparison views, and undo/restore support for planning iterations.

5. Domain-Specific Templates

Different project categories such as healthcare, education, e-commerce, or IoT can be supported using specialized stage questions and output templates.

6. Advanced Analytics

The system can provide planning completeness scores, requirement coverage metrics, and estimated project complexity based on blueprint content.

7. Direct Integration with Development Tools

Integration with Git repositories, issue trackers, and project management boards would allow generated TODO items and prompts to flow directly into execution environments.

In conclusion, BuildFlow AI is a successful AI-based project planning assistant that demonstrates both engineering value and future growth potential.

CHAPTER 9

REFERENCES

1. React Documentation:

Available at: https://react.dev/

2. Vite Documentation:

Available at: https://vite.dev/

3. Node.js Documentation:

Available at: https://nodejs.org/en/docs

4. Express.js Documentation:

Available at: https://expressjs.com/

5. Firebase Documentation:

Available at: https://firebase.google.com/docs

6. Cloud Firestore Documentation:

Available at: https://firebase.google.com/docs/firestore

7. Firebase Authentication Documentation:

Available at: https://firebase.google.com/docs/auth

8. OpenRouter Documentation:

Available at: https://openrouter.ai/docs/quickstart

9. MDN Web Docs - Fetch API:

Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

10. UML Basics and Modeling Concepts:

Standard software engineering textbook and lecture references used in academic practice.

CHAPTER 10

APPENDIX

This appendix includes representative planning prompts, sample AI response formats, and the blueprint structure used by BuildFlow AI.

10.1 Sample Prompt

Sample Prompt 1: Initial stage bootstrap prompt

Ask the first question for Stage 1: Idea and Goal.

Sample Prompt 2: Document generation context prompt

Generate a complete Business Requirements Document based on:
Project Title: BuildFlow AI
Idea: AI-based project planning assistant
Users: Students, beginner developers, startup teams
Features: Stage-based chat, blueprint storage, document generation
UI/UX: Dashboard, workspace, outputs, prompts
Tech Stack: React, Node.js, Firebase, OpenRouter

10.2 Sample AI Response (JSON format)


```
{
  "reply": "Your idea is clear. Who are the primary users and what planning difficulty are they facing today?",
  "stage_complete": false,
  "stage_summary": "",
  "stage_updates": {
    "stage1": "BuildFlow AI is an assistant that converts software ideas into structured planning outputs.",
    "stage2": "",
    "stage3": "",
    "stage4": "",
    "stage5": ""
  }
}
```

10.3 Sample Blueprint Structure

{
"stage1": "BuildFlow AI helps convert software project ideas into structured plans and documents.",
"stage2": "Primary users include students, small teams, and founders who need planning assistance.",
"stage3": "The system provides authentication, project storage, AI chat, blueprint generation, document outputs, and prompt packs.",
"stage4": "The interface uses a dashboard, stage navigator, chat panel, and side panels for outputs and prompts.",
"stage5": "The technology stack includes React frontend, Node/Express backend, Firebase, and OpenRouter AI integration."
}

10.4 Sample TODO-style breakdown derived from project outputs

Frontend Tasks:
- Build login and signup screens
- Implement dashboard and project cards
- Develop workspace with stage navigation and chat input

Backend Tasks:
- Create /api/chat and /api/generate-docs routes
- Implement AI service wrapper and response parser

Database Tasks:
- Store projects, blueprint, outputs, prompts, and settings in Firestore

Testing Tasks:
- Validate auth, chat flow, and output generation

Deployment Tasks:
- Configure environment variables and production build